

Competitive Programming

Section 41 Editorial: Euler Tour, LCA, and Binary Lifting

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
1.1	Problem-specific proof map	4
1.2	Detailed solution guide	4
2	A. Tree Distances	6
2.1	Restatement	6
2.2	Observations, trace, and algorithm	6
2.3	Correctness proof and complexity	6
2.4	Full C++ solution	6
2.5	Full Python solution	6
2.6	Tests that matter	7
3	B. Max Edge On Path	8
3.1	Restatement	8
3.2	Observations, trace, and algorithm	8
3.3	Correctness proof and complexity	8
3.4	Full C++ solution	8
3.5	Full Python solution	8
3.6	Tests that matter	9
4	C. Kth Node On Path	10
4.1	Restatement	10
4.2	Observations, trace, and algorithm	10
4.3	Correctness proof and complexity	10
4.4	Full C++ solution	10
4.5	Full Python solution	10
4.6	Tests that matter	11
5	D. Weighted Tree Distances	12
5.1	Restatement	12
5.2	Constraints and traps	12
5.3	Observations	12
5.4	Sample walk-through	12
5.5	Algorithm	13
5.6	Correctness proof	13
5.7	Complexity	13
5.8	C++ implementation notes	13
5.9	Python implementation notes	14
5.10	Full C++ solution	14
5.11	Full Python solution	14
5.12	Common wrong answers	15
5.13	Tests that matter	15
6	Testing discussion	16

1 Editorial overview

1.1 Problem-specific proof map

A. Tree Distances. Euler entry/exit times make a subtree a contiguous interval, while binary lifting raises two vertices to their lowest common ancestor. The identity $\text{dist}(u,v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}(u,v)]$ is exact because the shared root-to-LCA prefix is counted twice. Preprocessing costs $O(n \log n)$ and each query costs $O(\log n)$.

B. Max Edge On Path. Alongside $\text{up}[j][v]$, store the maximum edge on the 2^j -edge climb. Combining the same climbs used by LCA covers the two disjoint path halves, so their maximum is the answer. Queries remain $O(\log n)$.

C. K-th Node On Path. The path is the upward segment from u to the LCA followed by the reverse upward segment from v . Compare k with the first segment length and lift either endpoint by the corresponding number of edges; this covers each path vertex once.

1.2 Detailed solution guide

A. Tree Distances. Root the tree once and compute $\text{depth}[v]$ and $\text{up}[k][v]$, the 2^k -th ancestor of v . To answer $\text{lca}(u,v)$, first lift the deeper vertex until both depths match. Then lift both vertices from high powers down to low powers whenever their ancestors differ. Their parents are then the LCA. The distance formula is $\text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}]$.

B. Maximum Edge on a Path. Store one more table: $\text{mx}[k][v]$, the maximum edge weight on the path from v up by 2^k edges. When lifting a vertex, merge the corresponding mx value into the answer. The path from u to v splits into $u \rightarrow \text{lca}$ and $v \rightarrow \text{lca}$, so lifting both sides and taking the maximum over the chunks covers every path edge exactly once.

C. k-th Node on a Path. Let $w = \text{lca}(u,v)$. The path length in vertices is $\text{depth}[u] - \text{depth}[w] + 1 + \text{depth}[v] - \text{depth}[w]$. If k lies on the u to w part, jump $k-1$ steps upward from u . Otherwise the answer lies on the w to v part; convert it to a jump upward from v by counting how many vertices remain after position k .

Common pitfalls. Binary lifting uses powers of two, so allocate enough levels for the maximum n . For C, be explicit about whether k is 1-based; the local statement uses 1-based path positions.

This section has one preprocessing object: a binary-lifting table rooted at node 1. The three exercises reuse it with different query payloads.

- A uses only depth and up to compute $\text{lca}(u,v)$, then applies the distance formula.
- B stores $\text{mx}[k][v]$, the maximum edge on the upward jump of length 2^k . Every time the query lifts a vertex, it merges the corresponding maximum.
- C first computes the LCA and the total path length in nodes. If the requested k stays on the u side, jump upward from u ; otherwise convert it to a jump upward from v .

The proof pattern is the same in all three: the path is decomposed into disjoint upward power-of-two chunks, and the table stores the exact ancestor or aggregate for each such chunk.

2 A. Tree Distances

2.1 Restatement

Given a rooted tree and queries u, v , print the number of edges on the path.

2.2 Observations, trace, and algorithm

For n, q near $2e5$, parent climbing fails on a chain. Store $up[j][v]$, the 2^j ancestor, and depth. For sample nodes 4 and 5, $lca=2$, hence distance is $2+2-2*1=2$. Lift the deeper node, then descend powers from largest to zero, lifting both only when their ancestors differ; return their parent.

2.3 Correctness proof and complexity

$up[j][v]=up[j-1][up[j-1][v]]$ composes two exact half-jumps. Reverse-bit lifting leaves both vertices immediately below their lowest common ancestor, so their parent is the LCA. The two root paths are disjoint below it, proving the distance formula. Preprocessing is $O(n \log n)$, query time $O(\log n)$, memory $O(n \log n)$. QED.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,q;if(!(cin>>n>>q)) return 0; int LOG=1; while((1<<LOG)<=n+1) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n+1)); vector<int> depth(n+1);
    for(int v=2;v<=n;v++){cin>>up[0][v]; depth[v]=depth[up[0][v]]+1;}
    for(int k=1;k<LOG;k++) for(int v=1;v<=n;v++) up[k][v]=up[k-1][up[k-1][v]];
    auto lift=[&](int v,int d){for(int k=0;k<LOG;k++) if(d&(1<<k)) v=up[k][v]; return v;};
    auto lca=[&](int a,int b){if(depth[a]<depth[b]) swap(a,b); a=lift(a,depth[a]-depth[b]); if
    while(q--){int a,b;cin>>a>>b; int c=lca(a,b); cout<<depth[a]+depth[b]-2*depth[c]<<'\n';}
}
```

2.5 Full Python solution

```

import sys

def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2
    LOG=max(1,(n+1).bit_length())
    up=[[0]*(n+1) for _ in range(LOG)]; depth=[0]*(n+1)
    for v in range(2,n+1):
        p=data[idx]; idx+=1; up[0][v]=p; depth[v]=depth[p]+1
    for k in range(1,LOG):
        for v in range(1,n+1): up[k][v]=up[k-1][up[k-1][v]]
    def lift(v,d):
        bit=0
        while d:
            if d&1: v=up[bit][v]
            d//=2; bit+=1
        return v
    def lca(a,b):
        if depth[a]<depth[b]: a,b=b,a
        a=lift(a,depth[a]-depth[b])
        if a==b: return a
        for k in range(LOG-1,-1,-1):
            if up[k][a]!=up[k][b]: a=up[k][a]; b=up[k][b]
        return up[0][a]
    out=[]
    for _ in range(q):
        a,b=data[idx],data[idx+1]; idx+=2; c=lca(a,b)
        out.append(str(depth[a]+depth[b]-2*depth[c]))
    print("\n".join(out))
if __name__=="__main__": main()

```

2.6 Tests that matter

Test $u=v$, root queries, ancestors, a chain, a maximum edge at the LCA boundary, both path endpoints, and invalid k values.

3 B. Max Edge On Path

3.1 Restatement

Each edge to a parent has a weight. For each $u \ v$, print the maximum edge weight on the path.

3.2 Observations, trace, and algorithm

Carry $mx[j][v]$, the maximum edge on the same 2^j upward segment. Whenever a lift uses bit j , merge $mx[j][v]$ before moving v . Its recurrence is $\max(mx[j-1][v], mx[j-1][up[j-1][v]])$; merge both sides while finding LCA.

3.3 Correctness proof and complexity

The recurrence joins exactly adjacent half-segments. Lifting partitions the $u-v$ path into those segments, so their merged maximum is exact. Costs match A; the empty local path $u==v$ has maximum zero. QED.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,q;if(!(cin>>n>>q)) return 0; int LOG=1; while((1<<LOG)<=n+1) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n+1)), mx(LOG, vector<int>(n+1)); vector<int> depth
    for(int v=2;v<=n;v++){int p,w;cin>>p>>w; up[0][v]=p; mx[0][v]=w; depth[v]=depth[p]+1;}
    for(int k=1;k<LOG;k++) for(int v=1;v<=n;v++){int mid=up[k-1][v]; up[k][v]=up[k-1][mid]; mx
    auto lift=[&](int &v,int d){int ans=0; for(int k=0;k<LOG;k++) if(d&(1<<k)){ans=max(ans,mx[k
    while(q--){int a,b;cin>>a>>b; int ans=0; if(depth[a]<depth[b]) swap(a,b); ans=max(ans,lift
    }
```

3.5 Full Python solution

```

import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2; LOG=max(1,(n+1).bit_length())
    up=[[0]*(n+1) for _ in range(LOG)]; mx=[[0]*(n+1) for _ in range(LOG)]; depth=[0]*(n+1)
    for v in range(2,n+1):
        p,w=data[idx],data[idx+1]; idx+=2; up[0][v]=p; mx[0][v]=w; depth[v]=depth[p]+1
    for k in range(1,LOG):
        for v in range(1,n+1):
            mid=up[k-1][v]; up[k][v]=up[k-1][mid]; mx[k][v]=max(mx[k-1][v],mx[k-1][mid])
    def lift(v,d):
        ans=0; bit=0
        while d:
            if d&1: ans=max(ans,mx[bit][v]); v=up[bit][v]
            d//=2; bit+=1
        return v,ans
    out=[]
    for _ in range(q):
        a,b=data[idx],data[idx+1]; idx+=2; ans=0
        if depth[a]<depth[b]: a,b=b,a
        a,val=lift(a,depth[a]-depth[b]); ans=max(ans,val)
        if a!=b:
            for k in range(LOG-1,-1,-1):
                if up[k][a]!=up[k][b]:
                    ans=max(ans,mx[k][a],mx[k][b]); a=up[k][a]; b=up[k][b]
            ans=max(ans,mx[0][a],mx[0][b])
        out.append(str(ans))
    print("\n".join(out))
if __name__=="__main__": main()

```

3.6 Tests that matter

Test $u=v$, root and ancestor queries, a chain, maximum-edge boundaries, and both valid and invalid k values.

4 C. Kth Node On Path

4.1 Restatement

For query $u \ v \ k$, print the k -th node on the path from u to v , or -1 .

4.2 Observations, trace, and algorithm

Let $c=lca(u,v)$, $left=depth[u]-depth[c]+1$, and $total=depth[u]+depth[v]-2*depth[c]+1$. Reject invalid k . If $k \leq left$, return $lift(u,k-1)$; otherwise return $lift(v,total-k)$. On path 4,2,5 this returns 4,2,5 for $k=1,2,3$.

4.3 Correctness proof and complexity

The first left path nodes are exactly u -to- c ; the remaining nodes are v -to- c in reverse, so $total-k$ is the required upward distance from v . Time is $O(\log n)$ with the shared table. QED.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,q;if(!(cin>>n>>q)) return 0; int LOG=1; while((1<<LOG)<=n+1) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n+1)); vector<int> depth(n+1);
    for(int v=2;v<=n;v++){cin>>up[0][v]; depth[v]=depth[up[0][v]]+1;}
    for(int k=1;k<LOG;k++) for(int v=1;v<=n;v++) up[k][v]=up[k-1][up[k-1][v]];
    auto lift=[&](int v,int d){for(int k=0;k<LOG;k++) if(d&(1<<k)) v=up[k][v]; return v;};
    auto lca=[&](int a,int b){if(depth[a]<depth[b]) swap(a,b); a=lift(a,depth[a]-depth[b]); if
    while(q--){int u,v,k;cin>>u>>v>>k; int c=lca(u,v); int left=depth[u]-depth[c]+1,total=depth
    }
```

4.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
```

```

n,q=data[0],data[1]; idx=2; LOG=max(1,(n+1).bit_length())
up=[[0]*(n+1) for _ in range(LOG)]; depth=[0]*(n+1)
for v in range(2,n+1):
    p=data[idx]; idx+=1; up[0][v]=p; depth[v]=depth[p]+1
for k in range(1,LOG):
    for v in range(1,n+1): up[k][v]=up[k-1][up[k-1][v]]
def lift(v,d):
    for k in range(LOG):
        if d>>k & 1: v=up[k][v]
    return v
def lca(a,b):
    if depth[a]<depth[b]: a,b=b,a
    a=lift(a,depth[a]-depth[b])
    if a==b: return a
    for k in range(LOG-1,-1,-1):
        if up[k][a]!=up[k][b]: a=up[k][a]; b=up[k][b]
    return up[0][a]
out=[]
for _ in range(q):
    u,v,k=data[idx],data[idx+1],data[idx+2]; idx+=3; c=lca(u,v); left=depth[u]-depth[c]+1
    if k<1 or k>total: out.append("-1")
    elif k<=left: out.append(str(lift(u,k-1)))
    else: out.append(str(lift(v,total-k)))
print("\n".join(out))
if __name__=="__main__": main()

```

4.6 Tests that matter

Test $u==v$, root and ancestor queries, a chain, maximum-edge boundaries, and both valid and invalid k values.

5 D. Weighted Tree Distances

5.1 Restatement

Each edge has a signed weight. For every pair of nodes, print the sum of the weights on their unique tree path. This blind exercise requires the same LCA primitive as unweighted distance, but the depth count alone is no longer the quantity to combine.

5.2 Constraints and traps

With $n, q \leq 200,000$, walking a path per query is too slow on a chain. A path sum can reach about $2 * 10^{14}$, so use `long long` in C++. Edge weights may be negative: do not use a shortest-path interpretation, and do not initialize a path aggregate to zero except for the empty path.

5.3 Observations

Root the tree at node 1. Let `root_sum[v]` be the sum of edge weights on the root-to-`v` path. It is available while reading this parent-before-child input:

```
root_sum[v] = root_sum[parent[v]] + edge_weight[v].
```

For $c = \text{lca}(u, v)$, the two root paths share exactly the root-to- c prefix. Adding the two root sums counts that shared prefix twice, so remove it twice:

```
path_sum(u, v) = root_sum[u] + root_sum[v] - 2*root_sum[c].
```

The LCA table is the ordinary `up[t][v]` table. No edge aggregate table is needed because the prefix sums already summarize every root path.

5.4 Sample walk-through

For query (3,4), $\text{lca}(3,4)=1$. The root sums are 2 for node 3 and 16 for node 4, while the root sum at node 1 is zero:

```
2 + 16 - 2*0 = 18.
```

For (2,2), the LCA is node 2, so its root sum is added twice and subtracted twice. The result is zero, as the empty path should be.

5.5 Algorithm

```
read parent and edge weight for every node
compute depth[v], root_sum[v], and up[0][v]
build the binary-lifting table

for each query (u,v):
    c = lca(u,v)
    print root_sum[u] + root_sum[v] - 2*root_sum[c]
```

The LCA routine first equalizes depths, then attempts simultaneous jumps from largest power to smallest while the proposed ancestors differ.

5.6 Correctness proof

Lemma 1. $\text{root_sum}[v]$ equals the sum of weights on the root-to- v path.

Proof. The root has sum zero. Every other node appends exactly its parent edge to the already-correct parent path, so induction on input order proves the recurrence. $\square$

Lemma 2. The binary-lifting routine returns $\text{lca}(u, v)$.

Proof. Each table entry is an exact power-of-two ancestor by induction. After depth equalization, simultaneous largest-first jumps keep both nodes below their LCA. When no such jump remains, they are its children, so their common parent is the LCA. $\square$

Theorem. The algorithm prints the path-weight sum for every query.

Proof. By Lemma 2, c is the unique meeting point of the two root paths. By Lemma 1, $\text{root_sum}[u] + \text{root_sum}[v]$ contains every edge on the u - v path once and every edge on root-to- c twice. Subtracting $2*\text{root_sum}[c]$ removes exactly that duplicated prefix. $\square$

5.7 Complexity

```
Preprocessing:  $O(n \log n)$  time and memory
Each query:     $O(\log n)$  time
```

5.8 C++ implementation notes

Use `long long` for `root_sum` and the final expression. The `up` entries and depths remain ordinary integers. The parent-before-child input order means no recursive DFS is needed, avoiding recursion-depth issues on a chain.

5.9 Python implementation notes

Python integers naturally cover sums. Parse the input once, build output lines in a list, and use an iterative lifting loop; no recursion is needed.

5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; if (!(cin >> n >> q)) return 0;
    int log = 1; while ((1 << log) <= n) ++log;
    vector<vector<int>> up(log, vector<int>(n + 1));
    vector<int> depth(n + 1); vector<long long> root_sum(n + 1);
    for (int v = 2; v <= n; ++v) { long long w; cin >> up[0][v] >> w; depth[v] = depth[up[0][v]] + 1; root_sum[v] = root_sum[up[0][v]] + w; }
    for (int k = 1; k < log; ++k) for (int v = 1; v <= n; ++v) up[k][v] = up[k-1][up[k-1][v]];
    auto lift = [&](int v, int d) { for (int k = 0; k < log; ++k) if (d & (1 << k)) v = up[k][v]; return v; };
    auto lca = [&](int a, int b) { if (depth[a] < depth[b]) swap(a, b); a = lift(a, depth[a] - depth[b]); if (a == b) return a;
    while (q--) { int u, v; cin >> u >> v; int c = lca(u, v); cout << root_sum[u] + root_sum[v] - 2 * root_sum[c] << '\n'; }
}
```

5.11 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data: return
    n, q = data[0], data[1]; log = max(1, n.bit_length()); index = 2
    up = [[0] * (n + 1) for _ in range(log)]; depth = [0] * (n + 1); root_sum = [0] * (n + 1)
    for node in range(2, n + 1):
        parent, weight = data[index], data[index + 1]; index += 2
        up[0][node] = parent; depth[node] = depth[parent] + 1; root_sum[node] = root_sum[parent] + weight
    for bit in range(1, log):
        for node in range(1, n + 1): up[bit][node] = up[bit - 1][up[bit - 1][node]]
    def lift(node: int, distance: int) -> int:
        for bit in range(log):
            if distance >> bit & 1: node = up[bit][node]
        return node
    def lca(left: int, right: int) -> int:
        if depth[left] < depth[right]: left, right = right, left
        left = lift(left, depth[left] - depth[right])
        if left == right: return left
        for bit in range(log - 1, -1, -1):
```

```

        if up[bit][left] != up[bit][right]: left, right = up[bit][left], up[bit][right]
    return up[0][left]
output = []
for _ in range(q):
    left, right = data[index], data[index + 1]; index += 2; common = lca(left, right)
    output.append(str(root_sum[left] + root_sum[right] - 2 * root_sum[common]))
sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__": main()

```

5.12 Common wrong answers

Using edge count depth in place of weighted root sums.
 Subtracting root_sum[lca] once instead of twice.
 Using int for a long chain of large weights.
 Treating negative weights as an invalid input or shortest-path issue.
 Forgetting that a query from a node to itself has answer zero.

5.13 Tests that matter

u=v, including the root.
 An ancestor-descendant query.
 Negative edges on one branch.
 Two branches whose LCA is not the root.
 A 200,000-node chain with 200,000 endpoint queries.

6 Testing discussion

Run `CP_TARGET=solution python3 sections/41_euler_tour_lca_binary_lifting/check.py`. For random trees of at most 20 vertices, build a path by parent walks and use it as an oracle for all three exercises. This catches depth, aggregate-order, and one-based-rank mistakes more reliably than samples.

7 What to carry forward

Binary lifting turns an ancestor relation into a powers-of-two table. Once the exact jump invariant is established, LCA, path distance, path aggregates, and path indexing are small variations on the same controlled lifts.

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.